| | |
|---|---|
| **Status** | Finished |
| **Started** | Tuesday, 29 September 2026, 4:13 PM |
| **Completed** | Tuesday, 29 September 2026, 4:26 PM |
| **Duration** | 12 mins 7 secs |
| **Grade** | **10.00** out of 10.00 (**100**%) |

Question **1**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

**Description:**

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt | madam<br>arora<br>malayalam |

**Answer:** (penalty regime: 0 %)

```
import string

def find_palindromes(input_filename='input.txt', output_filename='output.txt'):
    # Try reading from input.txt, fallback to input1.txt if needed
    text = ""
    for filename in [input_filename, 'input1.txt']:
        try:
            with open(filename, 'r') as file:
                text = file.read()
                break
        except FileNotFoundError:
            continue

    words = text.split()
    palindromes = []
    seen = set()

    for word in words:
        # Strip punctuation and convert to lowercase for palindrome check
        cleaned = word.strip(string.punctuation).lower()
        if cleaned and cleaned == cleaned[::-1]:
            if cleaned not in seen:
                seen.add(cleaned)
                palindromes.append(word.strip(string.punctuation))

    # Write the unique palindrome words to output.txt
    with open(output_filename, 'w') as file:
        for p in palindromes:
            file.write(p + '\n')

if __name__ == '__main__':
    find_palindromes()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt | madam<br>arora<br>malayalam | madam<br>arora<br>malayalam | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

**Description:**

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

**Example:**

- **Input File Content:**

  apple orange apple banana apple orange

**Output:**

apple: 3
orange: 2
banana: 1

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `apple orange apple banana apple orange` | `apple: 3`<br>`banana: 1`<br>`orange: 2` |

**Answer:** (penalty regime: 0 %)

```
import sys
import string

def count_word_frequency():
    text = ""

    # 1. Try reading from 'input.txt' first
    try:
        with open('input.txt', 'r', encoding='utf-8') as file:
            text = file.read()
    except (FileNotFoundError, IOError):
        pass

    # 2. Fallback to standard input if 'input.txt' wasn't read successfully
    if not text.strip():
        text = sys.stdin.read()

    # 3. Clean punctuation, convert to lowercase, and count frequencies
    counts = {}
    for word in text.split():
        cleaned = word.strip(string.punctuation).lower()
        if cleaned:
            counts[cleaned] = counts.get(cleaned, 0) + 1

    # 4. Write results to 'output.txt' sorted alphabetically
    with open('output.txt', 'w', encoding='utf-8') as file:
        for word in sorted(counts.keys()):
            file.write(f"{word}: {counts[word]}\n")

if __name__ == '__main__':
    count_word_frequency()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r')`<br>`as file:`<br>`  text = file.read()`<br>`  print(text)` | `apple orange apple banana apple orange` | `apple: 3`<br>`banana: 1`<br>`orange: 2` | `apple: 3`<br>`banana: 1`<br>`orange: 2` | ✓ |
| ✓ | `with open('output.txt', 'r')`<br>`as file:`<br>`  text = file.read()`<br>`  print(text)` | `Hello world! Hello everyone. Welcome to the world of programming.` | `everyone: 1`<br>`hello: 2`<br>`of: 1`<br>`programming: 1`<br>`the: 1`<br>`to: 1`<br>`welcome: 1`<br>`world: 2` | `everyone: 1`<br>`hello: 2`<br>`of: 1`<br>`programming: 1`<br>`the: 1`<br>`to: 1`<br>`welcome: 1`<br>`world: 2` | ✓ |
| ✓ | `with open('output.txt', 'r')`<br>`as file:`<br>`  text = file.read()`<br>`  print(text)` | `One fish two fish Red fish blue fish` | `blue: 1`<br>`fish: 4`<br>`one: 1`<br>`red: 1`<br>`two: 1` | `blue: 1`<br>`fish: 4`<br>`one: 1`<br>`red: 1`<br>`two: 1` | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

**Description:**

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

**For example:**

| Input | Result |
|---|---|
| input1.txt<br>3 | Line three. |

**Answer:** (penalty regime: 0 %)

```
import sys

def read_specific_line():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # Parse filename and line number from input
    filename = 'input1.txt'
    line_num = 1

    if len(input_data) >= 2:
        filename = input_data[0].strip()
        try:
            line_num = int(input_data[1].strip())
        except ValueError:
            line_num = 1
    elif len(input_data) == 1:
        try:
            line_num = int(input_data[0].strip())
        except ValueError:
            filename = input_data[0].strip()

    # Try reading the file and fetching the specified line
    filenames_to_try = [filename, 'input1.txt', 'input.txt']
    for fname in filenames_to_try:
        try:
            with open(fname, 'r', encoding='utf-8') as file:
                lines = file.readlines()
                if 1 <= line_num <= len(lines):
                    print(lines[line_num - 1].strip())
                    return
        except FileNotFoundError:
            continue

if __name__ == '__main__':
    read_specific_line()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | input1.txt<br>3 | Line three. | Line three. | ✓ |
| ✓ | input2.txt<br>3 | Line C. | Line C. | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

**Description:**

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

**For example:**

| Input | Result |
|---|---|
| input2.txt | Total words: 14 |
| input3.txt | Total words: 15 |

**Answer:** (penalty regime: 0 %)

```
import sys

def count_total_words():
    # Read input from standard input (e.g., filename)
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    filename = input_data[0].strip()

    # Try reading the specified file, with graceful fallbacks
    text = ""
    filenames_to_try = [filename, 'input2.txt', 'input3.txt', 'input.txt', 'input1.txt']
    for fname in filenames_to_try:
        try:
            with open(fname, 'r', encoding='utf-8') as file:
                text = file.read()
                break
        except FileNotFoundError:
            continue

    # Split text into words and count them
    words = text.split()
    print(f"Total words: {len(words)}")

if __name__ == '__main__':
    count_total_words()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | input1.txt | Total words: 6 | Total words: 6 | ✓ |
| ✓ | input2.txt | Total words: 14 | Total words: 14 | ✓ |
| ✓ | input3.txt | Total words: 15 | Total words: 15 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

**Description:**

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

**Example:**

- **Input File Content:**

  **"Line one.**
  **Line two.**
  **Line three.**
  **Line four."**
  **2**

  **Updated line two.**

**Output:**

Line one.
Updated line two.
Line three.
Line four.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`2`<br>`Updated line two.` | `Line one.`<br>`Updated line two.`<br>`Line three.`<br>`Line four.` |

**Answer:** (penalty regime: 0 %)

```
import sys

def update_file_line():
    # Read all lines from standard input
    input_lines = sys.stdin.read().splitlines()
    if len(input_lines) < 3:
        return

    # Parse filename, line number, and new content
    filename = input_lines[0].strip()
    try:
        target_line_num = int(input_lines[1].strip())
    except ValueError:
        target_line_num = 1

    new_content = input_lines[2]

    # Try reading the source text file with graceful fallbacks
    file_lines = []
    filenames_to_try = [filename, 'input1.txt', 'input.txt']
    for fname in filenames_to_try:
        try:
            with open(fname, 'r', encoding='utf-8') as f:
                file_lines = f.readlines()
                break
        except FileNotFoundError:
            continue

    # Replace the specified line (1-indexed)
```

```
    if 1 <= target_line_num <= len(file_lines):
        # Preserve original line ending if present
        ends_with_newline = file_lines[target_line_num - 1].endswith('\n')
        file_lines[target_line_num - 1] = new_content + ('\n' if ends_with_newline else '')

    # Write the updated content to 'output.txt'
    with open('output.txt', 'w', encoding='utf-8') as f:
        f.writelines(file_lines)

if __name__ == '__main__':
    update_file_line()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt<br>2<br>Updated line two. | Line one.<br>Updated line two.<br>Line three.<br>Line four. | Line one.<br>Updated line two.<br>Line three.<br>Line four. | ✓ |
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input2.txt<br>2<br>Line B Updated. | Line A.<br>Line B Updated.<br>Line C. | Line A.<br>Line B Updated.<br>Line C. | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

**Description:**

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

**Example:**

- **Input File Content:**

  **"Line one.**
  **Line two.**
  **Line three.**
  **Line four."**
  **3**

**Output:**

Line one.
Line two.
eerht eniL.
Line four.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`3` | `Line one.`<br>`Line two.`<br>`eerht eniL.`<br>`Line four.` |

**Answer:** (penalty regime: 0 %)

```
import sys
import string

def reverse_line_content():
    # Read filename and target line number from standard input
    input_lines = sys.stdin.read().splitlines()
    if len(input_lines) < 2:
        return

    filename = input_lines[0].strip()
    try:
        target_line_num = int(input_lines[1].strip())
    except ValueError:
        target_line_num = 1

    # Read the content of the target text file with graceful fallbacks
    file_lines = []
    filenames_to_try = [filename, 'input1.txt', 'input2.txt', 'input.txt']
    for fname in filenames_to_try:
        try:
            with open(fname, 'r', encoding='utf-8') as f:
                file_lines = f.readlines()
                break
        except FileNotFoundError:
            continue

    if not file_lines:
        return

    # Convert 1-indexed line number to 0-indexed list position
    target_idx = target_line_num - 1
```

```
    if 0 <= target_idx < len(file_lines):
        line = file_lines[target_idx]
        has_newline = line.endswith('\n')
        content = line.rstrip('\r\n')

        # Reverse the text content while keeping trailing punctuation at the end
        if content and content[-1] in string.punctuation:
            punct = content[-1]
            core = content[:-1]
            reversed_content = core[::-1] + punct
        else:
            reversed_content = content[::-1]

        file_lines[target_idx] = reversed_content + ('\n' if has_newline else '')

    # Write only the updated file lines into 'output.txt'
    with open('output.txt', 'w', encoding='utf-8') as f:
        f.writelines(file_lines)

if __name__ == '__main__':
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt<br>3 | Line one.<br>Line two.<br>eerht eniL.<br>Line four. | Line one.<br>Line two.<br>eerht eniL.<br>Line four. | ✓ |
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input2.txt<br>2 | Line A.<br>B eniL.<br>Line C. | Line A.<br>B eniL.<br>Line C. | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

**Description:**

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

**Example:**

- **Input File Content:**

  **"Line one.**
  **Line two.**
  **Line three.**
  **Line four."**
  **2**

  **Updated line two.**

**Output:**

Line one.
Line three.
Line four.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt<br>2 | Line one.<br>Line three.<br>Line four. |

**Answer:** (penalty regime: 0 %)

```
import sys

def delete_specific_line():
    # Read filename and line number from standard input
    input_lines = sys.stdin.read().splitlines()
    if len(input_lines) < 2:
        return

    filename = input_lines[0].strip()
    try:
        target_line_num = int(input_lines[1].strip())
    except ValueError:
        target_line_num = 1

    # Read the text file lines with graceful fallbacks
    file_lines = []
    filenames_to_try = [filename, 'input1.txt', 'input.txt']
    for fname in filenames_to_try:
        try:
            with open(fname, 'r', encoding='utf-8') as f:
                file_lines = f.readlines()
                break
        except FileNotFoundError:
            continue

    # Convert 1-indexed line number to 0-indexed list position and delete
    target_idx = target_line_num - 1
    if 0 <= target_idx < len(file_lines):
        del file_lines[target_idx]

    # Write the updated lines to 'output.txt'
    with open('output.txt', 'w', encoding='utf-8') as f:
        f.writelines(file_lines)
```

```
34
35 if __name__ == '__main__':
36     delete_specific_line()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt<br>2 | Line one.<br>Line three.<br>Line four. | Line one.<br>Line three.<br>Line four. | ✓ |
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input2.txt<br>3 | Line A.<br>Line B. | Line A.<br>Line B. | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

**Description:**

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

**Example:**

- **Input File Content:**

  **"Line one.**
  **Line two.**
  **Line three.**
  **Line four."**
  **3**

  **Inserted line..**

**Output:**

Line one.
Line two.
Inserted line.
Line three.
Line four.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`3`<br>`Inserted line.` | `Line one.`<br>`Line two.`<br>`Inserted line.`<br>`Line three.`<br>`Line four.` |

**Answer:** (penalty regime: 0 %)

```
import sys

def insert_line_at_position():
    # Read all lines from standard input
    input_lines = sys.stdin.read().splitlines()
    if len(input_lines) < 3:
        return

    # Parse filename, line number, and new line content
    filename = input_lines[0].strip()
    try:
        target_line_num = int(input_lines[1].strip())
    except ValueError:
        target_line_num = 1

    new_content = input_lines[2].strip()

    # Read the source text file lines with graceful fallbacks
    file_lines = []
    filenames_to_try = [filename, 'input1.txt', 'input2.txt', 'input.txt']
    for fname in filenames_to_try:
        try:
            with open(fname, 'r', encoding='utf-8') as f:
                file_lines = [line.rstrip('\r\n') for line in f.readlines()]
                break
        except FileNotFoundError:
            continue
```

```
28
29      # Convert 1-indexed line number to 0-indexed list position
30      target_idx = target_line_num - 1
31      if target_idx < 0:
32          target_idx = 0
33      elif target_idx > len(file_lines):
34          target_idx = len(file_lines)
35
36      # Insert the new content at the exact position
37      file_lines.insert(target_idx, new_content)
38
39      # Write the updated lines to 'output.txt' with consistent newlines
40      with open('output.txt', 'w', encoding='utf-8') as f:
41          for i, line in enumerate(file_lines):
42              f.write(line + '\n')
43
44  if __name__ == '__main__':
45      insert_line_at_position()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt<br>3<br>Inserted line. | Line one.<br>Line two.<br>Inserted line.<br>Line three.<br>Line four. | Line one.<br>Line two.<br>Inserted line.<br>Line three.<br>Line four. | ✓ |
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input2.txt<br>4<br>Inserted line D. | Line A.<br>Line B.<br>Line C.<br>Inserted line D. | Line A.<br>Line B.<br>Line C.<br>Inserted line D. | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

**For example:**

| Input | Result |
|---|---|
| input1.txt | Longest word: containing |

**Answer:** (penalty regime: 0 %)

```
import sys
import string

def find_longest_word():
    # Read filename from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    filename = input_data[0].strip()

    # Read the text file with graceful fallbacks
    text = ""
    filenames_to_try = [filename, 'input1.txt', 'input.txt']
    for fname in filenames_to_try:
        try:
            with open(fname, 'r', encoding='utf-8') as f:
                text = f.read()
                break
        except FileNotFoundError:
            continue

    # Clean punctuation and extract words
    longest_word = ""
    max_length = 0

    for word in text.split():
        cleaned = word.strip(string.punctuation)
        if len(cleaned) > max_length:
            max_length = len(cleaned)
            longest_word = cleaned

    print(f"Longest word: {longest_word}")

if __name__ == '__main__':
    find_longest_word()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | input1.txt | Longest word: containing | Longest word: containing | ✓ |
| ✓ | input2.txt | Longest word: thousand | Longest word: thousand | ✓ |
| ✓ | input3.txt | Longest word: supercalifragilisticexpialidocious | Longest word: supercalifragilisticexpialidocious | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

**Description:**

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output1.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`output1.txt` | `This is the source file.`<br>`It contains multiple lines of text.`<br>`Here is another line.` |

**Answer:** (penalty regime: 0 %)

```
import sys

def copy_file_contents():
    # Read input lines for source and destination file names
    input_lines = sys.stdin.read().splitlines()
    tokens = [line.strip() for line in input_lines if line.strip()]

    src = tokens[0] if len(tokens) >= 1 else 'input1.txt'
    dest = tokens[1] if len(tokens) >= 2 else 'output1.txt'

    # Read the source file with graceful fallbacks
    content = ""
    filenames_to_try = [src, 'input1.txt', 'input.txt']
    for fname in filenames_to_try:
        try:
            with open(fname, 'r', encoding='utf-8') as f:
                content = f.read()
                break
        except FileNotFoundError:
            continue

    # Write the content to the destination file
    with open(dest, 'w', encoding='utf-8') as f:
        f.write(content)

if __name__ == '__main__':
    copy_file_contents()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output1.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`output1.txt` | `This is the source file.`<br>`It contains multiple lines of text.`<br>`Here is another line.` | `This is the source file.`<br>`It contains multiple lines of text.`<br>`Here is another line.` | ✓ |
| ✓ | `with open('output2.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input2.txt`<br>`output2.txt` | `Hello, world!`<br>`Python programming is amazing.`<br>`Let's copy this text to another file.` | `Hello, world!`<br>`Python programming is amazing.`<br>`Let's copy this text to another file.` | ✓ |
| ✓ | `with open('output3.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input3.txt`<br>`output3.txt` | `Single line.` | `Single line.` | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.